

- Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.
- Samuel L Smith, Andrew Brock, Leonard Berrada, and Soham De. Convnets match vision transformers at scale. *arXiv preprint arXiv:2310.16764*, 2023.
- Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *JMLR*, 2014.
- Wenfang Sun, Xinyuan Song, Pengxiang Li, Lu Yin, Yefeng Zheng, and Shiwei Liu. The curse of depth in large language models. *arXiv preprint arXiv:2502.05795*, 2025.
- Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2024.
- Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *CVPR*, 2016.
- Hidenori Tanaka and Daniel Kunin. Noether’s learning dynamics: Role of symmetry breaking in neural networks. *NeurIPS*, 2021.
- Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. *NeurIPS*, 2021.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023a.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023b.
- Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *NeurIPS*, 2017.
- Yuxin Wu and Kaiming He. Group normalization. In *ECCV*, 2018.
- Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *CVPR*, 2017.
- Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *ICML*, 2020.
- Jingjing Xu, Xu Sun, Zhiyuan Zhang, Guangxiang Zhao, and Junyang Lin. Understanding and improving layer normalization. *NeurIPS*, 2019.
- An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.
- Shuangfei Zhai, Tatiana Likhomanenko, Etai Littwin, Dan Busbridge, Jason Ramapuram, Yizhe Zhang, Jiatao Gu, and Joshua M Susskind. Stabilizing transformer training by preventing attention entropy collapse. In *ICML*, 2023.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. *NeurIPS*, 2019.
- Hongyi Zhang, Yann N Dauphin, and Tengyu Ma. Fixup initialization: Residual learning without normalization. *arXiv preprint arXiv:1901.09321*, 2019.
- Pan Zhang, Xiaoyi Dong, Yuhang Zang, Yuhang Cao, Rui Qian, Lin Chen, Qipeng Guo, Haodong Duan, Bin Wang, Linke Ouyang, et al. Internlm-xcomposer-2.5: A versatile large vision language model supporting long-contextual input and output. *arXiv preprint arXiv:2407.03320*, 2024.

Appendix

A Experimental Settings

Supervised image classification. For all supervised classification experiments on ImageNet-1K, we follow the training recipes from ConvNeXt (Meta Research, a). For ConvNeXt-B and ConvNeXt-L, we use the original hyperparameters without modification. ViT-B and ViT-L models use the same hyperparameters as ConvNeXt-B, except that for ViT-L, the beta parameters for AdamW are set to (0.9, 0.95), and the stochastic depth rates are set to 0.1 for ViT-B and 0.4 for ViT-L.

Diffusion models. We use the official implementation (Meta Research, c) for training all DiT models. We find that the default learning rate is suboptimal for the models considered in this paper. To address this, we conduct a simple learning rate search with the LN models and apply the tuned learning rates directly to the DyT models. We also observe that the zero initialization negatively affects the performance of DyT models. Therefore, we retain the zero initialization for LN models but remove the zero initialization for DyT models.

Large Language Models. In our implementation of LLaMA models (Touvron et al., 2023a,b; Dubey et al., 2024) with DyT, we introduce an additional learnable scalar parameter immediately after the embedding layer, before any Transformer blocks. We initialize it to the square root of the model embedding dimension $\sqrt{d}$. Without this scaling scalar, we find that the magnitudes of model activations at the beginning of training are too small, and the training struggles to progress. The issue is mitigated by incorporating a learnable scalar, and the model can converge normally. This addition of a scalar is similar to the original Transformer (Vaswani et al., 2017) design, which uses a fixed scalar of the same value at the same position.

We train all our LLaMA models on the Pile dataset (Gao et al., 2020). We use the codebase from FMS-FSDP (Foundation Model Stack), which provides a default training recipe for the 7B model that closely follows the LLaMA 2 paper (Touvron et al., 2023b). We maintain the learning rate at the default 3e-4 for 7B and 13B and 1.5e-4 for 34B and 70B, in line with LLaMA 2. The batch size is set to 4M tokens and each model is trained on a total of 200B tokens.

For evaluation, we test the pretrained models on 15 zero-shot commonsense reasoning tasks from lm-eval (Gao et al.): anli_r1, anli_r2, anli_r3, arc_challenge, arc_easy, boolq, hellaswag, openbookqa, piqa, record, rte, truthfulqa_mc1, truthfulqa_mc2, wic, and winogrande. The selection closely follows that of OpenLLaMA (Geng and Liu, 2023). We report the average performance across all tasks.

Self-supervised learning in speech. For both wav2vec 2.0 models, we retain the first group normalization layer from the original architecture, as it functions primarily as data normalization to handle the unnormalized input data. We use the official implementation (Meta Research, e) without modifying hyperparameters for both the Base and Large models. We report the final validation loss.

Other tasks. For all other tasks, MAE (He et al., 2022), DINO (Caron et al., 2021), HyenaDNA (Nguyen et al., 2024) and Caduceus (Schiff et al., 2024), we directly use the publicly released code (Meta Research, d,b; HazyResearch; Kuleshov Group), without hyperparameter tuning, for both models with LN and DyT.

B Hyperparameters

We present additional experiments to evaluate the impact of hyperparameter tuning, specifically focusing on the learning rate and initialization of α for all non-LLM models.

Tuning learning rate. Table 12 summarizes performance comparisons between models trained with original versus tuned learning rates. Results indicate that tuning the learning rate provides only modest performance improvements for DyT models. This suggests that the original hyperparameters, initially optimized for LN models, are already well-suited for DyT models. This observation underscores the inherent similarity between the DyT and LN models.

model	LN (original)	DyT (original)	LN (tuned)	DyT (tuned)
ViT-B	82.3% (4e-3)	82.5% (4e-3)	-	82.8% (6e-3)
ViT-L	83.1% (4e-3)	83.6% (4e-3)	-	-
ConvNeXt-B	83.7% (4e-3)	83.7% (4e-3)	-	-
ConvNeXt-L	84.3% (4e-3)	84.4% (4e-3)	-	-
MAE ViT-B	83.2% (2.4e-3)	83.2% (2.4e-3)	-	83.7% (3.2e-3)
MAE ViT-L	85.5% (2.4e-3)	85.4% (2.4e-3)	-	85.8% (3.2e-3)
DINO ViT-B (patch size 16)	83.2% (7.5e-4)	83.4% (7.5e-4)	83.3% (1e-3)	-
DINO ViT-B (patch size 8)	84.1% (5e-4)	84.5% (5e-4)	-	-
DiT-B	64.9 (4e-4)	63.9 (4e-4)	-	-
DiT-L	45.9 (4e-4)	45.7 (4e-4)	-	-
DiT-XL	19.9 (4e-4)	20.8 (4e-4)	-	-
wav2vec 2.0 Base	1.95 (5e-4)	1.95 (5e-4)	-	1.94 (6e-4)
wav2vec 2.0 Large	1.92 (3e-4)	1.91 (3e-4)	-	-
HyenaDNA	85.2% (6e-4)	85.2% (6e-4)	-	-
Caduceus	86.9% (8e-3)	86.9% (8e-3)	-	-

Table 12 Performance comparison between original and tuned learning rates for LN and DyT models. Results show that tuning learning rates provide only modest performance improvements for DyT models, suggesting that the default hyperparameters optimized for LN models are already well-suited for DyT models. Entries marked with “-” indicate no performance gain over the original learning rate. The values in parentheses represent the learning rate used.

Tuning initial value of α . We also investigate the effects of optimizing α_0 for DyT models, as presented in Table 13. Findings show only minor performance enhancements for select models when α_0 is tuned, indicating that the default initial value ($\alpha_0 = 0.5$) generally achieves near-optimal performance.

Model	LN	DyT ($\alpha_0 = 0.5$)	DyT (tuned)
ViT-B	82.3%	82.5%	82.6% ($\alpha_0 = 1.0$)
ViT-L	83.1%	83.6%	-
ConvNeXt-B	83.7%	83.7%	-
ConvNeXt-L	84.3%	84.4%	-
MAE ViT-B	83.2%	83.2%	83.4% ($\alpha_0 = 1.0$)
MAE ViT-L	85.5%	85.4%	-
DINO ViT-B (patch 16)	83.2%	83.4%	-
DINO ViT-B (patch 8)	84.1%	84.5%	-
DiT-B	64.9	63.9	-
DiT-L	45.9	45.7	-
DiT-XL	19.9	20.8	-
wav2vec 2.0 Base	1.95	1.95	-
wav2vec 2.0 Large	1.92	1.91	1.90 ($\alpha_0 = 1.0$)
HyenaDNA	85.2%	85.2%	-
Caduceus	86.9%	86.9%	-

Table 13 Impact of tuning the α_0 in DyT models. Optimizing α_0 from the default value ($\alpha_0 = 0.5$) yields only minor performance gains for select DyT models, implying the default initialization already achieves near-optimal performance. Entries marked with “-” indicate no improvement over the default α_0 .

C Efficiency of DyT

We benchmark the LLaMA 7B model with RMSNorm or DyT by measuring the total time required for 100 forward passes (inference) and 100 forward-backward passes (training) on a single sequence of 4096 tokens. We first follow the officially recommended LLaMA setup and load the model from [Hugging Face](#) without applying any performance optimizations. Table 14 reports the time taken for RMSNorm and DyT layers, as